

CSE321 - Assignment #1

Han SeungHo (20211356)

UNIST

South Korea

hsh5405@unist.ac.kr

1 Problem Statement

The purpose of this project is to implement and evaluate tree-based index structures that are commonly used in database systems. In this assignment, I implemented three index structures: a B-tree, a B⁺-tree, and a B*⁺-tree. Each tree was implemented from scratch in Python, and the order parameter d was made configurable so that the effect of fan-out could be tested experimentally.

The input data consist of 100,000 student records loaded from a CSV file. Each record contains a student ID, name, gender, GPA, height, and weight. To follow the simplified storage model required by the project manual, the records are stored in an in-memory array. The student ID is used as the search key, while the array index is used as the record identifier (RID). Therefore, the index structures do not store complete student records directly; instead, they store keys and RIDs, and the actual record values are accessed through the storage array when an analytical query is evaluated.

This storage model is important because it separates the index from the actual table-like data. The tree only decides which RID is associated with a key, and the in-memory array is responsible for storing the full student tuple. In the code, this is implemented by appending each parsed Student object to STORAGE; the loop index produced by `enumerate(STORAGE)` is then inserted into each tree as the RID. This design follows the manual requirement that the student ID must be the key and the array index must serve as the RID.

```
for rid, student in enumerate(STORAGE):
    tree.insert(student.sid, rid)
```

The implementation supports insertion, point search, range query, and deletion workloads. The main experimental objective is to compare the three tree structures in terms of execution time, number of node splits, number of nodes, and final node utilization. The experiments were conducted for three different order values, $d = 3$, $d = 5$, and $d = 10$, in order to observe how the fan-out parameter changes both performance and structural overhead.

The project manual also requires that the implementation be written from scratch, without using library implementations of tree-based index structures. My implementation uses only basic Python data structures such as lists and object references. The imported modules in `main.py` are used only for CSV loading, random workload generation, timing, and path handling. All search, insertion, deletion, split, redistribution, merge, and range scan logic is implemented directly in `implement.py`.

For reproducibility, the source code, dataset, and execution guide should be provided in a public repository before submission. The repository includes `main.py`, `implement.py`, `addition.py`, `student.csv` and a `README.md` file describing how to run the experiments. The public repository URL is: <https://github.com/hsh5405/Database-System>

2 Fundamental Index Structure (B-tree and B⁺-tree)

2.1 B-tree

The B-tree implementation uses a node structure containing a leaf flag, a sorted list of keys, a list of RIDs, and a list of child pointers. For a tree of order d , each node can contain at most $d - 1$ keys. The minimum number of keys is implemented as $\lfloor (d - 1)/2 \rfloor$ except for the root, which may contain fewer keys. In this implementation, internal nodes may store both keys and RIDs, so a successful search can terminate at an internal node.

The order parameter is interpreted as the maximum number of children in an internal node. Therefore, the maximum number of keys is $d - 1$. For B-tree and B*⁺-tree nodes, the minimum number of keys is implemented as $(d - 1)/2$, which is equivalent to $\lceil d/2 \rceil - 1$ for the order values used in the experiment. The root is treated as an exception: after deletion, if an internal root has no key, its only child is promoted as the new root.

The core node structure is intentionally simple. Keys and RIDs are stored at the same list index, so whenever a key moves during split, borrowing, or merging, its RID must move together.

```
BNode:
    is_leaf
    keys[]
    rids[]
    children[]
```

Insertion is performed recursively. When a key reaches a leaf node, it is inserted into the sorted key list together with its RID. If a child overflows, the middle key is promoted to the parent, and the remaining keys are divided into two nodes. When the root overflows, a new root is created and the old root becomes its first child. The implementation counts these split events in order to measure structural overhead.

The split policy is a standard promote-middle policy. The middle key and its RID are moved to the parent, while the left and right parts remain in two child nodes.

```
mid = len(child.keys) // 2
sep_key = child.keys[mid]
new.keys = child.keys[mid+1:]
child.keys = child.keys[:mid]
parent.keys.insert(index, sep_key)
```

Deletion first searches for the target key. If the key is found in an internal node, it is replaced by its predecessor from the left subtree, and the predecessor is then deleted recursively. If a child node has fewer than the minimum number of keys after deletion, the tree first attempts to borrow a key from a sibling. If borrowing is not possible, the node is merged with a sibling and the separator key from the parent is moved down. If the root becomes empty after

deletion, the height is reduced by replacing the root with its only child.

The underflow recovery procedure is organized as borrowing before merging. This is used to maintain the B-tree occupancy constraint while avoiding unnecessary node removal.

```
if left sibling has extra key:
    borrow from left
elif right sibling has extra key:
    borrow from right
else:
    merge with a sibling
```

Range query in the B-tree is implemented by recursively visiting the relevant parts of the tree and collecting RIDs whose keys are between the lower and upper bounds. Since records may appear in internal nodes as well as leaf nodes, the traversal must check both node keys and child subtrees.

For this reason, B-tree range query is not implemented as a leaf scan. It performs a depth-first traversal over the relevant key interval, because an RID may be located in an internal node.

2.2 B⁺-tree

The B⁺-tree implementation separates routing information from record references. Internal nodes contain only separator keys and child pointers, while all RIDs are stored in leaf nodes. In addition, leaf nodes are connected by a linked pointer, which allows range queries to scan consecutive leaves after reaching the first relevant leaf.

The minimum occupancy rule is also separated between internal nodes and leaf nodes. Internal nodes use `min_keys_inner = (d-1)//2`, while leaves use `min_keys_leaf = d//2`. This distinction is necessary because leaf nodes store actual key-RID pairs, whereas internal nodes only store routing separators.

The B⁺-tree node therefore has an additional next pointer. Internal nodes do not use `rids`, but the field is kept in the node class so that leaf nodes can store RID values using the same object type.

```
BPNode:
    is_leaf
    keys[]
    children[]
    rids[]    # leaf only
    next     # leaf link
```

Search in the B⁺-tree always continues until a leaf node is reached. When a search key is equal to a separator key in an internal node, the implementation routes the search to the right child, because the actual RID is stored in the leaf level. Leaf splitting keeps the promoted separator key in the new right leaf and inserts a copy of that key into the parent. Internal node splitting promotes the middle separator key upward.

The leaf split uses copy-up semantics. The first key of the new right leaf is copied to the parent, but the same key remains in the leaf because all real RIDs must stay at the leaf level.

```
mid = (d + 1) // 2
new.keys = child.keys[mid:]
child.keys = child.keys[:mid]
new.next = child.next
```

```
child.next = new
parent.keys.insert(index, new.keys[0])
```

Range query first descends to the leaf that may contain the lower bound, then follows the linked leaves until the key exceeds the upper bound. This directly matches the manual requirement that linked leaf pointers should support efficient range scans.

```
move down to leaf containing low
while leaf is not None:
    collect keys in [low, high]
    stop when key > high
    leaf = leaf.next
```

The deletion procedure removes the key and RID from the leaf level. If a leaf or internal node underflows, the implementation first attempts redistribution with a sibling. If redistribution is not possible, it merges nodes and updates parent separator keys. In leaf merges, the linked leaf pointer is also updated so that subsequent range scans remain valid.

The deletion code has separate recovery logic for leaf and internal underflow. For leaf nodes, borrowing updates the corresponding parent separator, and merging reconnects the next pointer. For internal nodes, recovery moves separator keys and child pointers but does not move RIDs, because internal nodes in the B⁺-tree do not store RIDs.

3 Advanced Index Structures (B*-tree)

The B*-tree was implemented as an extension of the B-tree structure. It uses the same basic node representation as the B-tree, including sorted keys, RIDs, child pointers, and a leaf flag. Therefore, search and range query follow the same logical behavior as in the B-tree: a point search may terminate at an internal node, and a range query recursively checks relevant internal keys and child subtrees.

In the code, `BSTree` inherits from `BTree`. This means that the public behavior of `STATUS`, `search`, `range_query`, and the deletion helper functions is reused from the B-tree unless explicitly overridden. The insertion path is overridden because the manual specifically requires B*-tree overflow handling through sibling redistribution and 2-to-3 splitting.

The main difference appears in the insertion overflow policy. In a standard B-tree, an overflowing node is split immediately. In the B*-tree implementation, when a child overflows, the tree first checks whether its left or right sibling has free space. If such a sibling exists, the keys of the sibling, the parent separator, and the overflowing node are redistributed without creating a new node. This policy is intended to increase node utilization and reduce unnecessary splits.

The implementation makes this policy explicit in the overflow handler. It tries redistribution with the left sibling first, then with the right sibling, and only creates a new node when redistribution cannot be performed.

```
if left has space:
    redistribute(left, child)
elif right has space:
    redistribute(child, right)
else:
    split_2to3(full_sibling, child)
```

If neither sibling has enough free space, the implementation performs a 2-to-3 split. The overflowing node and one sibling are combined with the parent separator and then redistributed into three nodes. Two separator keys are placed in the parent. This policy creates one additional node from two full nodes, rather than immediately splitting one full node into two half-filled nodes. As a result, the B⁺-tree is expected to use fewer nodes and maintain higher utilization than the standard B-tree, although insertion can become slower because redistribution and 2-to-3 splitting require more key movement.

Root overflow is handled separately. Since the root has no sibling, redistribution and 2-to-3 splitting cannot be applied to root overflow. In that case, the implementation uses a normal B-tree-style split to create a new root. This is not used for ordinary non-root overflow, where the B⁺ policy is applied first.

For the 2-to-3 split, the implementation first concatenates all keys from the left node, the parent separator, and the right node. Then it chooses two separator positions and redistributes the remaining keys into three nodes.

```
all = left.keys + [parent_key] + right.keys
s1 = len(all) // 3
s2 = 2 * len(all) // 3
left.keys = all[:s1]
mid.keys = all[s1+1:s2]
right.keys = all[s2+1:]
```

For deletion, the B⁺-tree uses the same underflow recovery procedure as the B-tree implementation: it attempts borrowing first and performs a 2-to-1 merge when borrowing is not possible. This is a simplification compared with the more specialized theoretical B⁺-tree deletion policy, which may use a 3-to-2 merge to preserve high occupancy after deletion. I considered this simplification acceptable within the scope of this project because the project manual explicitly requires sibling redistribution and 2-to-3 splitting for B⁺-tree insertion, while the deletion requirement focuses on maintaining structural integrity through underflow handling or merging. In the experiment, the B⁺-tree still shows high node utilization after construction, mainly because its insertion policy keeps nodes more densely filled than the standard B-tree. Therefore, in this implementation, the most important distinction of the B⁺-tree is the insertion overflow handling policy.

This distinction is also visible in the split count. Compared with the B-tree, the B⁺-tree reduces split counts by about 16.08% at $d = 3$, 17.90% at $d = 5$, and 20.34% at $d = 10$. These reductions support that the redistribution-first policy is working as intended during insertion.

4 Experimental Results and Analysis

4.1 Experimental Setup

All experiments were executed on 100,000 student records. The records were first loaded into an in-memory array, and each tree stored the student ID as the key and the array index as the RID. The same workload was applied to all three tree structures for each order value $d = 3$, $d = 5$, and $d = 10$. To improve result stability, each tree-order case was repeated five times, and the reported execution times are averages over the five runs.

The experimental constants were fixed in `main.py` so that every tree was tested under the same condition. The random seed was also fixed, so the point search keys and deletion keys were reproducible across different tree implementations.

```
ORDER_VALUES = [3, 5, 10]
RANDOM_SEED = 42
SEARCH_COUNT = 10000
DELETE_COUNT = 2000
RANGE_LOW = 202000000
RANGE_HIGH = 202099999
```

The insertion workload inserted all 100,000 records into an initially empty tree. After insertion, I measured the total insertion time, number of splits, number of nodes, and node utilization. Utilization was computed consistently as $\text{key count} / (\text{node count} \times (d-1))$ over all nodes. For the B⁺-tree, this value is a structural occupancy metric rather than the number of stored records, because internal nodes store separator keys but no RIDs. The point search workload used 10,000 randomly selected student IDs with a fixed random seed. The range query workload used the key interval `[202000000, 202099999]` and then calculated the average GPA and average height of male students in the returned RID set. Finally, the deletion workload removed 2,000 keys selected from the search workload. The deletion operation was applied only to the index; the original storage array remained unchanged.

Execution time was measured with `time.perf_counter()` and converted to milliseconds. For point search, the table reports the average total time for 10,000 queries across five runs. Since the number of queries is identical for all trees, the mean retrieval time required by the manual is obtained by dividing the reported search time by 10,000.

As shown in Table 1, increasing d reduces structural overhead for all three trees. For example, the B-tree split count decreases from 74,357 at $d = 3$ to 16,381 at $d = 10$, and the B⁺-tree split count decreases from 116,452 to 17,939. This behavior is expected because a larger order allows each node to store more keys, which reduces tree height and decreases the frequency of overflow.

Table 1 also shows that the B⁺-tree consistently achieves the highest node utilization. Its utilization is 80.112799%, 82.803392%, and 85.110005% for $d = 3$, $d = 5$, and $d = 10$, respectively. This is higher than the B-tree utilization, which remains around 67–68%, and also higher than the B⁺-tree utilization, which remains around 71%. This result matches the purpose of the B⁺-tree insertion policy: redistribution and 2-to-3 splitting keep nodes more densely filled. However, this higher utilization does not produce the fastest insertion time in my five-run average. The B⁺-tree insertion time is slower than the B-tree for all three order values, which suggests that the additional redistribution and 2-to-3 split operations introduce enough CPU and list-manipulation overhead to outweigh the reduced split count in this Python implementation.

In Table 1, the B⁺-tree produces more split events than the B-tree for every tested order. This is partly because the B⁺-tree keeps every RID at the leaf level and copies separator keys into internal nodes, so a leaf split does not remove the separator key from the leaf. Therefore, the same dataset can produce more structural nodes than the B-tree even though the B⁺-tree later benefits from linked leaves during range scans.

Table 1: Insertion and structural metrics after inserting 100,000 records, averaged over five runs.

Tree	d	Insert time (ms)	Splits	Nodes	Utilization (%)
B-tree	3	1801.458380	74357	74371	67.230507
B*-tree	3	2141.103480	62400	62412	80.112799
B ⁺ -tree	3	2712.102580	116452	116466	71.525595
B-tree	5	1040.563940	36767	36776	67.979117
B*-tree	5	1337.915760	30184	30192	82.803392
B ⁺ -tree	5	1209.315180	46799	46808	71.718510
B-tree	10	731.836920	16381	16387	67.804425
B*-tree	10	1036.465040	13049	13055	85.110005
B ⁺ -tree	10	747.067540	17939	17945	71.487570

Table 2: Search, range query, and deletion performance, averaged over five runs.

Tree	d	Search time (ms)	Range time (ms)	Delete time (ms)
B-tree	3	96.802520	18.417940	43.110180
B*-tree	3	84.844260	17.122540	32.725580
B ⁺ -tree	3	98.593180	15.521020	32.225180
B-tree	5	77.526420	13.155500	33.808340
B*-tree	5	71.762620	11.077060	25.477140
B ⁺ -tree	5	67.430700	9.478740	21.693080
B-tree	10	67.044020	9.360580	23.268280
B*-tree	10	66.504200	9.558420	24.451840
B ⁺ -tree	10	56.576060	6.725720	20.538360

Table 3: Analytical result of the fixed range query.

Metric	Value
Returned records	14316
Male records	7214
Average male GPA	3.278453
Average male height	174.058082

For point search, Table 2 shows that all trees successfully found all 10,000 sampled keys. The search time generally decreases as d increases because the tree height becomes smaller. The B*-tree gives the best search time for $d = 3$, while the B⁺-tree is the fastest at $d = 5$ and $d = 10$. Although a B-tree search can terminate at an internal node, the actual runtime is also affected by node occupancy, tree height, and the cost of traversing Python lists inside each node.

For range query, Table 2 shows that the B⁺-tree is the fastest for every tested order. This result is consistent with the implementation design because the B⁺-tree first locates the starting leaf and then scans adjacent leaves through linked pointers. In contrast, the B-tree and B*-tree range queries must recursively traverse relevant subtrees and also check keys stored in internal nodes. The range query returned 14,316 records for all tree types. Among these records, 7,214 were male students, with an average GPA of 3.278453 and an average height of 174.058082, as summarized in Table 3. The identical analytical result across all tree structures provides evidence that the range query returned consistent RID sets.

For deletion, Table 2 reports that all experiments deleted 2,000 keys. The B⁺-tree is the fastest at every tested order in the five-run average. This is consistent with its leaf-level storage design,

although deletion time is still influenced by underflow repair, borrowing, merging, and parent separator updates. The B*-tree is faster than the B-tree at $d = 3$ and $d = 5$, but slightly slower at $d = 10$. This is not a contradiction of the theoretical insertion advantage of B*-trees, because my B* deletion recovery uses the simpler B-tree-style borrowing and 2-to-1 merging policy. After deletion, the trees are still able to maintain valid search paths by repairing underfull nodes and reducing the root height when the root becomes empty. Overall, the results indicate that larger fan-out improves many operations, while each tree structure has a different trade-off: the B*-tree improves space utilization after insertion, the B⁺-tree is especially effective for range queries and deletion in this workload, and the standard B-tree provides a simpler baseline with competitive insertion time.

Some performance trends are not purely theoretical because the implementation was written in Python and stores node keys in list objects. As a result, searching inside a node and inserting or deleting keys inside a node also take time. The measured execution time may also change depending on the computer environment, memory behavior, dataset distribution, insertion order, and query pattern. Therefore, I focus more on the relative trends observed in this experiment than on the exact absolute times. In a different environment or with a different dataset, the precise timing gaps may change, even if structural trends such as split count, node count, and utilization remain similar.

5 Additional Experiments

5.1 Motivation

After implementing the required B-tree, B*-tree, and B⁺-tree, I became interested in whether the same order should necessarily be used for both internal nodes and leaf nodes in a B⁺-tree. A standard B⁺-tree uses one order d , so internal nodes and leaf nodes are controlled by the same fan-out parameter. However, their roles are different. Internal nodes mainly route searches to the correct subtree, while leaf nodes store the actual key-RID pairs and are connected by linked pointers for range scans. Therefore, I tested whether assigning different orders to internal nodes and leaf nodes could change the performance trade-off.

5.2 Idea of B⁺-tree Design: Layered B⁺-tree

For this additional experiment, I implemented a layered B⁺-tree in `addition.py`. The implementation keeps the basic B⁺-tree design from `implement.py`: searches always descend to a leaf, all RIDs are stored only in leaves, and range queries scan leaf nodes through next pointers. The difference is that the maximum number of keys is selected by node type. Internal nodes use $d_i - 1$ keys, while leaf nodes use $d_l - 1$ keys.

```
max_keys_inner = di - 1
max_keys_leaf = dl - 1
max_keys_for(node):
    leaf -> max_keys_leaf
    internal -> max_keys_inner
```

5.3 Experimental Setup

I compared three plain B⁺-trees with $d = 10$, $d = 20$, and $d = 30$ against two layered B⁺-trees: ($d_i = 10, d_l = 30$) and ($d_i = 30, d_l = 10$). I selected the range from 10 to 30 as a moderate order range where the B⁺-tree was already reasonably stable, rather than using very small orders dominated by frequent splits or very large orders dominated by long in-node scans. Each case was repeated 10 times, and I report the average values.

5.4 Results and Analysis

As shown in Table 4, all five cases returned the same 14,316 records for the range query, so the additional experiment did not change query correctness. The utilization values were 71.487570%, 70.059233%, and 69.964457% for the plain B⁺-trees with $d = 10$, $d = 20$, and $d = 30$, respectively. For the layered cases, utilization was computed by summing the actual capacity of each node type: internal nodes used $d_i - 1$ capacity and leaf nodes used $d_l - 1$ capacity. Under this definition, the utilization values were 69.964457% for ($d_i = 10, d_l = 30$) and 71.566446% for ($d_i = 30, d_l = 10$).

For the plain B⁺-tree, Table 4 shows that increasing d from 10 to 20 improved insertion, range query, and deletion time. This is consistent with the structural metrics: splits decreased from 17,939 to 8,072, and nodes decreased from 17,945 to 8,077. However, increasing d further to 30 did not monotonically improve all operations. Although the number of splits and nodes continued to decrease, insertion, search, and deletion became slower than at $d = 20$. This suggests that a larger node capacity reduces tree height and split frequency, but it also increases the amount of linear scanning and

list manipulation inside each Python node. In this implementation, $d = 20$ appears to be a more balanced plain B⁺-tree setting than either $d = 10$ or $d = 30$.

The layered cases in Table 4 show a clearer difference. The tree with $d_i = 10$ and $d_l = 30$ outperformed the tree with $d_i = 30$ and $d_l = 10$ in all measured time metrics. It also used far fewer nodes: 5,731 nodes compared with 16,223 nodes. This result is reasonable because the leaf nodes store the actual key-RID pairs and dominate the number of nodes in a B⁺-tree. When d_l is large, each leaf can hold more records, which reduces leaf splits and reduces the number of leaf-to-leaf hops during range queries. On the other hand, when d_i is small, many more leaf nodes are created even if the internal-node order is large.

In this workload, the best average performance came from the layered B⁺-tree with $d_i = 10$ and $d_l = 30$. It achieved the fastest search, range query, and deletion times among the five tested cases, while its insertion time was also competitive. I do not interpret this as a general rule that leaf order should always be larger than internal order. Rather, the result suggests that for this dataset, workload, and Python list-based implementation, increasing leaf capacity can be more useful than increasing internal routing fan-out. Since B⁺-tree point search always ends at a leaf and range query is mostly performed through the leaf chain, tuning the leaf order separately may provide a practical optimization opportunity.

Overall, this additional experiment suggests that a layered B⁺-tree design can be beneficial when the workload heavily depends on leaf-level behavior. In particular, assigning a larger order to leaf nodes may reduce leaf splits and improve range-scan efficiency. However, the result should be understood as implementation-, dataset-, and workload-dependent rather than as a universal conclusion. A different programming language, memory layout, dataset distribution, key insertion order, or query pattern could shift the best choice of d_i and d_l . Still, this experiment shows that separating internal-node order and leaf-node order can increase the design choices available when selecting a tree structure for a particular environment and workload.

Finally, I found that B-family tree algorithms are more complicated to implement than they first appear, especially because each split, merge, redistribution, etc. must preserve the tree structure correctly. Although this process was challenging, it was valuable to implement the structures step by step and test their behavior through actual experiments. In future assignments, if the project conditions allow it, I would also like to explore more advanced ideas beyond this simple structural variation, such as applying quantum algorithms like Grover's algorithm to database problems.

Table 4: Additional experiment: plain and layered B⁺-tree comparison, averaged over 10 runs.

Tree	Insert (ms)	Search (ms)	Range (ms)	Delete (ms)	Splits	Nodes
B ⁺ ($d = 10$)	536.790240	37.913940	4.974550	16.063820	17939	17945
B ⁺ ($d = 20$)	479.256880	39.286220	4.149290	15.211920	8072	8077
B ⁺ ($d = 30$)	516.288090	42.774400	4.126030	16.793480	5167	5171
Layered (10,30)	503.539510	36.984540	4.112550	14.939170	5725	5731
Layered (30,10)	624.172670	48.412550	5.239380	20.694070	16218	16223